

LAPORAN JOBSHEET 15: AUTHREGISTER(DATABASEINTEGRATION) MATKUL PEMOGRAMAN BERBASIS NETWORK AHMAD DZUL FADHLI HANNAN | TI3F | 2341720106

C. LANGKAH PRAKTIKUM

Bagian 1 – Membuat Register View

Buat folder pada views dengan nama register dan tambahkan 2 file yaitu index.tsx

dan register.module.scss

Modifikasi file index.tsx (pada folder views/auth/register/index.tsx)

```
pemograman_framework - index.tsx
1 import Link from "next/link";
2 import style from "../../auth/register/register.module.scss";
3
4 const TampilanRegister = () => {
5   return(
6     <div className={style.register}>
7       <h1 className={style.register__title}>Halaman Register</h1>
8       <Link href="/auth/login">Ke Halaman Login</Link>
9     </div>
10   );
11 };
12
13 export default TampilanRegister;
```

Buka file register.tsx pada folder auth/register.tsx

Modifikasi file register.tsx (pada folder pages/auth/register.tsx)

```
pemograman_framework - register.jsx
1 import Link from "next/link";
2 import TampilanRegister from "@views/auth/register"
3
4 const halamanRegister = () => {
5   return(
6     <>
7       <TampilanRegister />
8     </>
9   );
10 };
11
12 export default halamanRegister;
```

Modifikasi register.module.scss

pemograman_framework - register.module.scss

```
1 .register{
2   padding: 40px;
3 }
4
5 .register__title {
6   font-size: 28px;
7   font-weight: bold;
8 }
```

Tambahkan form inputan pada file index.tsx (pada folder views/auth/register/index.tsx) Form berisi:

- Email
- Fullname
- Password
- Button Register

Kode keseluruahannya sebagai berikut :

pemograman_framework - index.tsx

```
1 import Link from "next/link";
2 import style from "../../auth/register/register.module.scss";
3
4 const TampilanRegister = () => {
5   return (
6     <div className={style.register}>
7       <h1 className={style.register__title}>Halaman Register</h1>
8       <div className={style.register__form}>
9         <form action="">
10           <div>
11             <div>
12               <label
13                 htmlFor="email"
14                 className={style.register__form_item_label}>
15                 Email
16               </label>
17               <input
18                 type="email"
19                 id="email"
20                 name="email"
21                 placeholder="Email"
22                 className={style.register__form_item_input}>
23             </div>
24             <div>
25               <div>
26                 <label
27                   htmlFor="fullName"
28                   className={style.register__form_item_label}>
29                   Full Name
30                 </label>
31                 <input
32                   type="text"
33                   id="fullName"
34                   name="fullName"
35                   placeholder="Full Name"
36                   className={style.register__form_item_input}>
37               </div>
38               <div>
39                 <label
40                   htmlFor="password"
41                   className={style.register__form_item_label}>
42                   Password
43                 </label>
44                 <input
45                   type="password"
46                   id="password"
47                   name="password"
48                   placeholder="Password"
49                   className={style.register__form_item_input}>
50               </div>
51             </div>
52             <div>
53               <button type="submit" className={style.register__form_item_button}>
54                 Register
55               </button>
56             </div>
57           </form>
58           <br />
59           <p className={style.register__form_item_text}>
60             Sudah punya akun? <Link href="/auth/login">Ke Halaman Login</Link>
61           </p>
62         </div>
63       </div>
64     </div>
65   );
66 };
67
68 export default TampilanRegister;
```

Modifikasi register.module.scss

```
pemograman_framework - register.module.scss

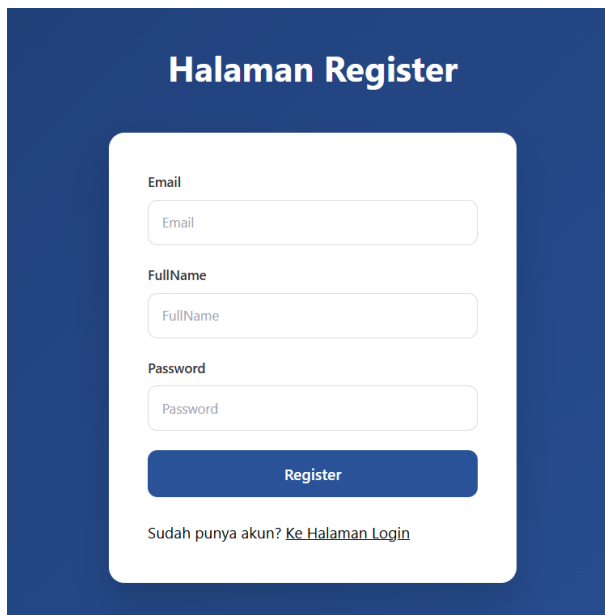
1  .register {
2    min-height: 100vh;
3    display: flex;
4    flex-direction: column; /* <-- penting */
5    align-items: center;
6    justify-content: center;
7    background: linear-gradient(135deg, #1e3c72, #2a5298);
8    padding: 20px;
9
10   &__title {
11     width: 100%;
12     text-align: center;
13     font-size: 36px;
14     font-weight: 700;
15     color: #ffffff;
16     margin-bottom: 40px;
17   }
18
19   &__form {
20     background: #ffffff;
21     width: 100%;
22     max-width: 420px;
23     padding: 40px;
24     border-radius: 16px;
25     box-shadow: 0 20px 40px rgba(0, 0, 0, 0.15);
26
27     &__item {
28       display: flex;
29       flex-direction: column;
30       margin-bottom: 20px;
31
32       &__label {
33         font-size: 14px;
34         font-weight: 600;
35         color: #333;
36         margin-bottom: 8px;
37       }
38
39       &__input {
40         padding: 12px 14px;
41         font-size: 14px;
42         border: 1px solid #ddd;
43         border-radius: 10px;
44         transition: all 0.3s ease;
45
46         &:focus {
47           border-color: #2a5298;
48           outline: none;
49           box-shadow: 0 0 0 3px rgba(42, 82, 152, 0.2);
50         }
51       }
52
53       &__button {
54         width: 100%;
55         padding: 12px;
56         font-weight: 600;
57         color: #fff;
58         background: #2a5298;
59         border: none;
60         border-radius: 10px;
61         cursor: pointer;
62         transition: 0.3s ease;
63
64         &:hover {
65           background-color: #1e3c72;
66           transform: translateY(-2px);
67         }
68       }
69     }
70   }
```

```

71
72 /* Animation */
73 @keyframes fadeIn {
74   from {
75     opacity: 0;
76     transform: translateY(15px);
77   }
78   to {
79     opacity: 1;
80     transform: translateY(0);
81   }
82 }
83
84 /* Responsive */
85 @media (max-width: 768px) {
86   .register {
87     &__form {
88       padding: 30px 20px;
89     }
90     &__title {
91       font-size: 26px;
92     }
93   }
94 }
95

```

Jalankan browsernya <http://localhost:3000/auth/register> sehingga tampilan sebagai berikut



The screenshot shows a registration page with a dark blue background. At the top, the title 'Halaman Register' is displayed in white. Below the title is a white rounded rectangle containing the registration form. The form has three input fields: 'Email', 'FullName', and 'Password'. Below these fields is a blue 'Register' button. At the bottom of the form, there is a link that says 'Sudah punya akun? [Ke Halaman Login](#)'.

Bagian 2 – Membuat API Register

Buka file `servicefirebase.ts` pada folder `src/Utils/db` dan modifikasi


```
pemograman_framework - servicefirebase.ts

1  import {
2    getFirestore,
3    collection,
4    getDocs,
5    Firestore,
6    getDoc,
7    doc,
8    query,
9    addDoc,
10   where,
11 } from "firebase/firestore";
12 import app from "../firebase";
13
14 const db = getFirestore(app);
15
16 export async function retrieveProducts(collectionName: string) {
17   const snapshot = await getDocs(collection(db, collectionName));
18   const data = snapshot.docs.map((doc) => ({
19     id: doc.id,
20     ...doc.data(),
21   }));
22   return data;
23 }
24
25 export async function retrieveDataById(collectionName: string, id: string) {
26   const snapshot = await getDoc(doc(db, collectionName, id));
27   const data = snapshot.data();
28   return data;
29 }
30
31 export async function retrieveLapars(collectionName: string) {
32   const snapshot = await getDocs(collection(db, collectionName));
33   const data = snapshot.docs.map((doc) => ({
34     id: doc.id,
35     ...doc.data(),
36   }));
37   return data;
38 }
39
40 export async function signUp(
41   userData: {
42     email: string;
43     fullname: string;
44     password: string;
45   },
46   callback: Function,
47 ) {
48   const q = query(
49     collection(db, "users"),
50     where("email", "==", userData.email),
51   );
52   const querySnapshot = await getDocs(q);
53   const data = querySnapshot.docs.map((doc) => ({
54     id: doc.id,
55     ...doc.data(),
56   }));
57   // console.log("Query result:", data);
58   if (data.length > 0) {
59     // user belum ada -> boleh daftar
60     // await addDoc(collection(db, "users"), userData);
61     // console.log("User registered:", data);
62     callback({
63       status: "success",
64       message: "User registered successfully",
65     });
66   } else {
67     callback({
68       status: "error",
69       message: "User already exists",
70     });
71   }
72 }
73
```

Buat file register.ts pada folder api

Modifikasi file register.ts

```
pemograman_framework - register.ts
1 // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2 import type { NextApiResponse, NextApiRequest } from "next";
3 import { signUp } from "@utils/db/servicefirebase";
4 type Data = {
5   name: string;
6   alamat: string;
7 };
8 export default async function handler(
9   req: NextApiRequest,
10  res: NextApiResponse<Data>,
11 ) {
12   if(req.method === "POST"){
13     await signUp(req.body, (result: {status: string, message: string}) => {
14       if(result.status){
15         res.status(200).json({ name: result.message, alamat: "" });
16       } else {
17         res.status(400).json({ name: result.message, alamat: "" });
18       }
19     });
20   }
21   else{
22     res.status(405).json({ name: "Method Not Allowed", alamat: "" });
23   }
24 }
```

Modifikasi index.tsx pada folder register (tambahkan beberapa code)

```
pemograman_framework - index.tsx
3 import { useState } from "react";
4 import { useRouter } from "next/router";
5
6 const TampilanRegister = () => {
7   const [isLoading, setIsLoading] = useState(false);
8   const { push } = useRouter();
9   const [error, setError] = useState("");
10  const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
11    event.preventDefault();
12    const form = event.currentTarget;
13    const formData = new FormData(event.currentTarget);
14    const email = formData.get("email") as string;
15    const fullname = formData.get("FullName") as string;
16    const password = formData.get("Password") as string;
17    const response = await fetch("/api/register", {
18      method: "POST",
19      headers: {
20        "Content-Type": "application/json",
21      },
22      body: JSON.stringify({
23        email,
24        fullname,
25        password,
26      }),
27    });
28    // const result = await response.json();
29    // console.log(result);
30    if (response.status === 200) {
31      form.reset();
32      // event.currentTarget.reset();
33      setIsLoading(false);
34      push("/auth/login");
35    } else {
36      setIsLoading(false);
37      setError(
38        response.status === 400 ? "User already exist" : "An error occurred",
39      );
40    }
41  };
42  return (
43    <div className={style.register}>
44      <h1 className={style.register__title}>Halaman Register</h1>
45      <div className={style.register__form}>
46        <form onSubmit={handleSubmit}>
```

Buka browser <http://localhost:3000/auth/register> isikan data dan klik register. Jika berhasil maka akan masuk ke menu login

Halaman Register

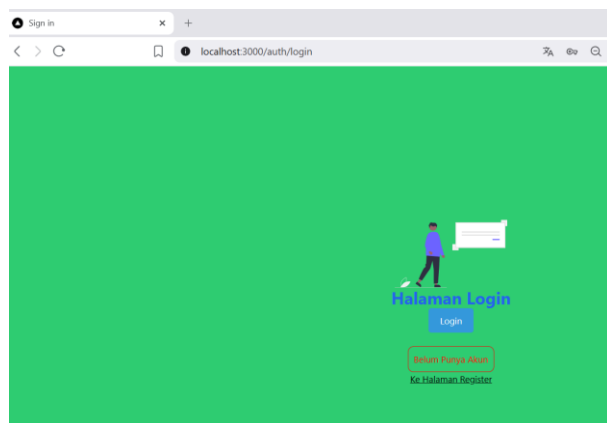
Email

FullName

Password

[Register](#)

Sudah punya akun? [Ke Halaman Login](#)



Benar

Bagian 3 – Install bcrypt

`npm install bcrypt --force`

```
• > npm install bcrypt --force
npm warn using --force Recommended protections disabled.

added 3 packages, and audited 489 packages in 4s

158 packages are looking for funding
  run `npm fund` for details

5 vulnerabilities (2 moderate, 3 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
❏ > █
```

npm install --save-dev @types/bcrypt --force

```
• > npm install --save-dev @types/bcrypt --force
npm warn using --force Recommended protections disabled.

added 1 package, and audited 490 packages in 2s

158 packages are looking for funding
  run `npm fund` for details

5 vulnerabilities (2 moderate, 3 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
❏ > █
```

Buka file servicefirebase.ts pada folder src/utils/db dan modifikasi

```

pemograman_framework - servicefirebase.ts

1  import {
2    getFirestore,
3    collection,
4    getDocs,
5    Firestore,
6    getDoc,
7    doc,
8    query,
9    addDoc,
10   where,
11 } from "firebase/firestore";
12 import app from "../firebase";
13 import bcrypt from "bcrypt";
14
15 const db = getFirestore(app);
16
17 export async function retrieveProducts(collectionName: string) {
18   const snapshot = await getDocs(collection(db, collectionName));
19   const data = snapshot.docs.map((doc) => ({
20     id: doc.id,
21     ...doc.data(),
22   }));
23   return data;
24 }
25
26 export async function retrieveDataById(collectionName: string, id: string) {
27   const snapshot = await getDoc(doc(db, collectionName, id));
28   const data = snapshot.data();
29   return data;
30 }
31
32 export async function retrieveLapars(collectionName: string) {
33   const snapshot = await getDocs(collection(db, collectionName));
34   const data = snapshot.docs.map((doc) => ({
35     id: doc.id,
36     ...doc.data(),
37   }));
38   return data;
39 }
40
41 export async function signUp(
42   userData: {
43     email: string;
44     fullname: string;
45     password: string;
46     role?: string;
47   },
48   callback: Function,
49 ) {
50   const q = query(
51     collection(db, "users"),
52     where("email", "==", userData.email),
53   );
54   const querySnapshot = await getDocs(q);
55   const data = querySnapshot.docs.map((doc) => ({
56     id: doc.id,
57     ...doc.data(),
58   }));
59   // console.log("Query result:", data);
60   if (data.length > 0) {
61     // user belum ada -> boleh daftar
62     // await addDoc(collection(db, "users"), userData);
63     // console.log("User registered:", data);
64     callback({
65       status: "success",
66       message: "User registered successfully",
67     });
68   } else {
69     userData.password = await bcrypt.hash(userData.password, 10);
70     userData.role = "user";
71     await addDoc(collection(db, "users"), userData)
72       .then(() => {
73         callback({
74           status: "success",
75           message: "User registered successfully",
76         });
77       })
78       .catch((error) => {
79         callback({
80           status: "error",
81           message: "User already exists",
82         });
83       });
84   }
85 }
86

```

Jalankan browser <http://localhost:3000/auth/register> dan input data setelah itu klik register

Halaman Register

Email

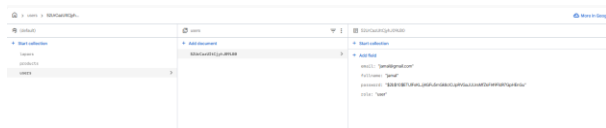
FullName

Password

Register

Sudah punya akun? [Ke Halaman Login](#)

Buka pada firebase jika berhasil maka data register akan masuk



Jika user memasukkan data yang sama sistem tidak akan memproses tetapi permasalahannya user memasukkan data yang sama tidak ada pemberitahuan pada layar maka dari itu perlu ada perubahan pada code index.tsx pada folder views/auth/register

```

pemograman_framework - index.tsx
43 <div className={style.register}>
44   {error && <p className={style.register__error}>{error}</p>}
45   <h1 className={style.register__title}>Halaman Register</h1>
46

```

```

pemograman_framework - index.tsx
97 <button
98   type="submit"
99   className={style.register__form__item__button}
100  disabled={isLoading}
101 >
102   {isLoading ? "Loading..." : "Register"}
103 </button>

```

Ubah jadi email

```

pemograman_framework - index.tsx
38 response.status === 400 ? "Email already exist" : "An error occurred",

```

Modifikasi juga pada register.module.scss

```
pemograman_framework - register.module.scss

71 &__error {
72   color: red;
73   font-size: 14px;
74   margin-top: 8px;
75 }
```

Perbaiki fungsi signUp pada utils/db/servicefirebase.ts

```
pemograman_framework - servicefirebase.ts

62 if (data.length > 0) {
63   // user sudah ada
64   return callback({
65     status: "error",
66     message: "User already exists",
67   });
68 }
69
70 try {
71   userData.password = await bcrypt.hash(userData.password, 10);
72   userData.role = "user";
73
74   await addDoc(collection(db, "users"), userData);
75
76   callback({
77     status: "success",
78     message: "User registered successfully",
79   });
80 } catch (error) {
81   callback({
82     status: "error",
83     message: "Failed to register user",
84   });
85 }
```

Dan api/register

```
pemograman_framework - register.ts

14 if(result.status === "success"){
```

Jika berhasil maka hasilnya seperti berikut

Email already exist

Halaman Register

Email

Full Name

Password

Register

Sudah punya akun? [Ke Halaman Login](#)

Tambahkan loading dengan menambahkan kode pada index.tsx

```
11 setError("");  
12 setIsLoading(true);
```

Jika berhasil maka hasilnya akan muncul loading saat klik register

Email already exist

Halaman Register

Email

jamal@gmail.com

Full Name

jamal

Password

....

Loading...

Sudah punya akun? [Ke Halaman Login](#)

Benar

D. PENGUJIAN

Uji 1 – Register Baru

Input:

- Email baru

Halaman Register

Email

jamaludin@gmail.com

Full Name

jamal

Password

....

Register

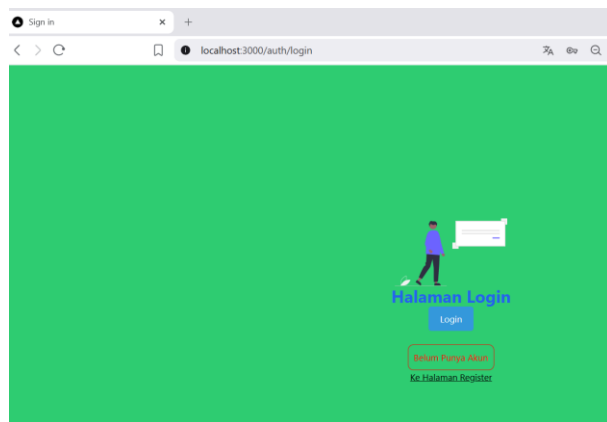
Sudah punya akun? [Ke Halaman Login](#)

Hasil:

- 
- The screenshot shows the AWS IAM console 'Users' page. The 'Users' list table has one entry: 'root' with a status of 'Active'. The 'root' user is selected, and the 'root' user details page is displayed on the right. The details page shows the user's name as 'root', the user's email as 'root@amazon.com', and the user's password as 'root'.

- ```
password: "$2b$10$z2eMrl4lxBR5o6v.1rMgW.B/7J4UgxnOe75AUYNhjvpjSnG8oMMlq"
```

- Redirect ke login



Input:

- # Halaman Register

Email

Full Name

Password

Register

Sudah punya akun? [Ke Halaman Login](#)

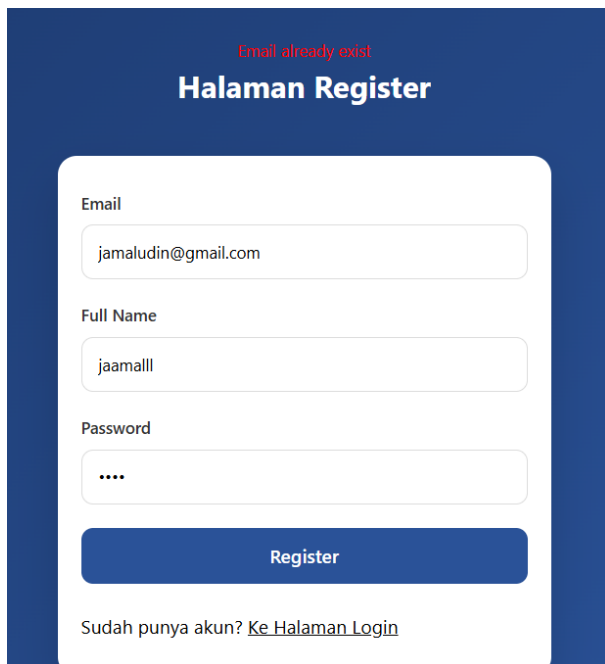
Hasil:

- Error 400

```
console.log("Response status:", response.status); x3 b 'Response status:' | 400 ⌚ b '
```

Benar

- Message: Email already exists



The screenshot shows a registration form on a dark blue background. At the top, a red error message "Email already exist" is displayed. Below it is the title "Halaman Register". The form itself is a white rounded rectangle containing three input fields: "Email" with the value "jamaludin@gmail.com", "Full Name" with the value "jaamalll", and "Password" with masked characters "....". A blue "Register" button is positioned below the inputs. At the bottom of the form, there is a link: "Sudah punya akun? [Ke Halaman Login](#)".

Benar

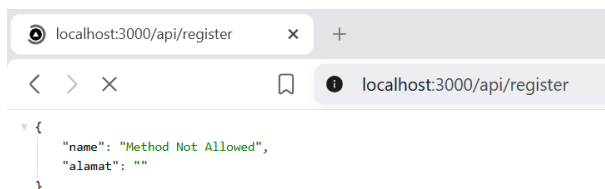
### Uji 3 – Method GET

Akses:

/api/register

Hasil:

- 405 Method Not Allowed



Benar

## E. STRUKTUR DATABASE (FIRESTORE)

Collection: users

Field:

| Field     | Tipe            |
|-----------|-----------------|
| fullName  | string          |
| email     | string          |
| password  | string (hashed) |
| role      | string          |
| createdAt | timestamp       |

## G. TUGAS PRAKTIKUM

### 1. Implementasikan register terhubung database.

**Jawab:** Sudah pakai firebase berbasis NoSQL

### 2. Tambahkan validasi:

o Email wajib

o Password minimal 6 karakter

```
pemograman_framework - index.tsx

18 const password = formData.get("password") as string;
19 if (!email) {
20 setIsLoading(false);
21 setError("Email is required");
22 return
23 }
24 if (password.length < 6) {
25 setIsLoading(false);
26 setError("Password must be at least 6 characters");
27 return;
28 }
29 const response = await fetch("/api/register", {
```

Hasil

Email

Email is required

## Halaman Register

Email

Full Name

Password

Register

Sudah punya akun? [Ke Halaman Login](#)

### Password

Password must be at least 6 characters

## Halaman Register

Email

Full Name

Password

Register

Sudah punya akun? [Ke Halaman Login](#)

### 3. Tambahkan role default "member".

Modifikasi utils/db/servicefirebase.ts

```
pemograman_framework - servicefirebase.ts

72 userData.role = "member";
```

#### 4. Tampilkan pesan error di UI.

Sudah

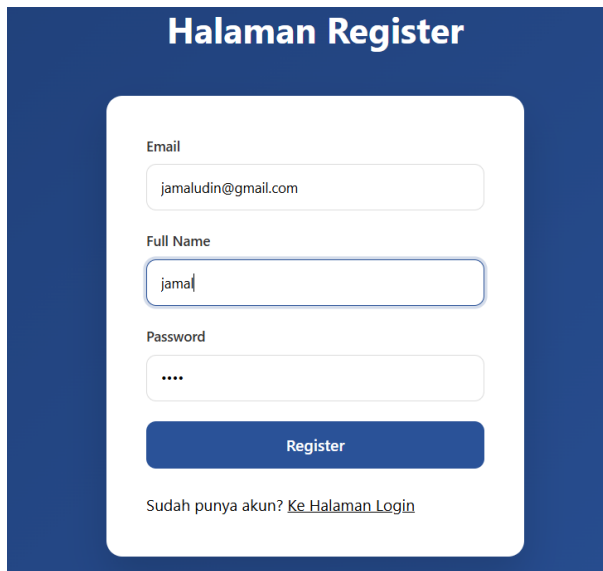
```
pemograman_framework - index.tsx

56 <div className={style.register}>
57 {error && <p className={style.register_error}>{error}</p>} /* Pesan ERROR */
58 <h1 className={style.register_title}>Halaman Register</h1>
```

#### 5. Screenshot hasil:

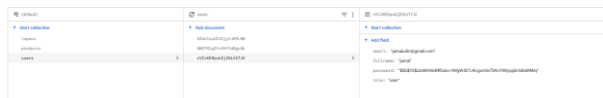
o Register sukses

- Email baru



Hasil:

- Data tersimpan di Firestore

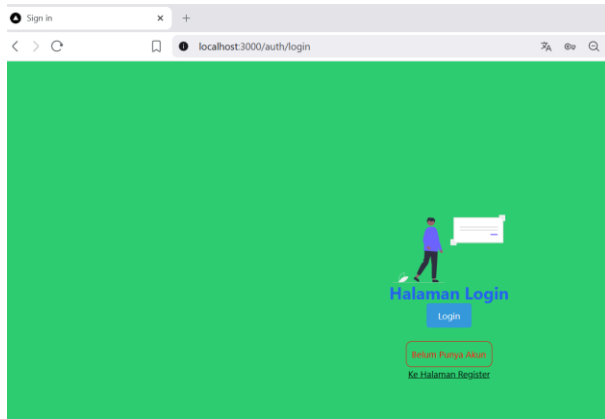


- Password ter-hash

password: "\$2b\$10\$z2eMr14lxBR5o6v.1rMgW.B/7J4UgxnOe75AUYNhjvpj\$SnG8oMMlq"

Benar

- Redirect ke login



o Email sudah ada

Hasil:

- Error 400

```
console.log("Response status:", response.status); x3 b 'Response status:' | 400 ⌚ b '
```

Benar

- Message: Email already exists

Email already exist

## Halaman Register

Email

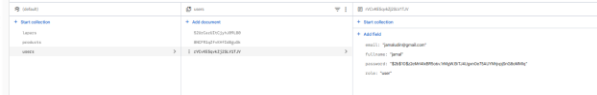
Full Name

Password

[Register](#)

Sudah punya akun? [Ke Halaman Login](#)

o Database Firestore



## H. PERTANYAAN ANALISIS

1. Mengapa password harus di-hash?

Agar password tersimpan aman dan tidak dapat dibaca langsung jika database bocor.

2. Apa perbedaan addDoc dan setDoc?

addDoc membuat dokumen baru dengan ID otomatis, sedangkan setDoc membuat atau menimpa dokumen dengan ID yang ditentukan.

3. Mengapa perlu validasi method POST?

Untuk memastikan API hanya menerima request yang sesuai dan mencegah perubahan data lewat method lain.

4. Apa risiko jika email tidak dicek unik?

Bisa terjadi duplikasi akun, kebingungan autentikasi, dan potensi masalah keamanan.

5. Apa fungsi role pada user?

Menentukan hak akses atau level izin user dalam aplikasi.

## I. OUTPUT YANG DIHARAPKAN

Mahasiswa menghasilkan:

- ✓ Form registrasi berfungsi
- ✓ API POST berjalan
- ✓ Email unik tervalidasi
- ✓ Password ter-hash
- ✓ Data tersimpan di Firestore
- ✓ Error tampil di UI
- ✓ Redirect ke login





**LAPORAN JOBSHEET 16: IMPLEMENTASI LOGIN DATABASE & MULTI-ROLE MATKUL  
PEMOGRAMAN BERBASIS NETWORK  
AHMAD DZUL FADHLI HANNAN | TI3F | 2341720106**

---

**C. LANGKAH PRAKTIKUM**

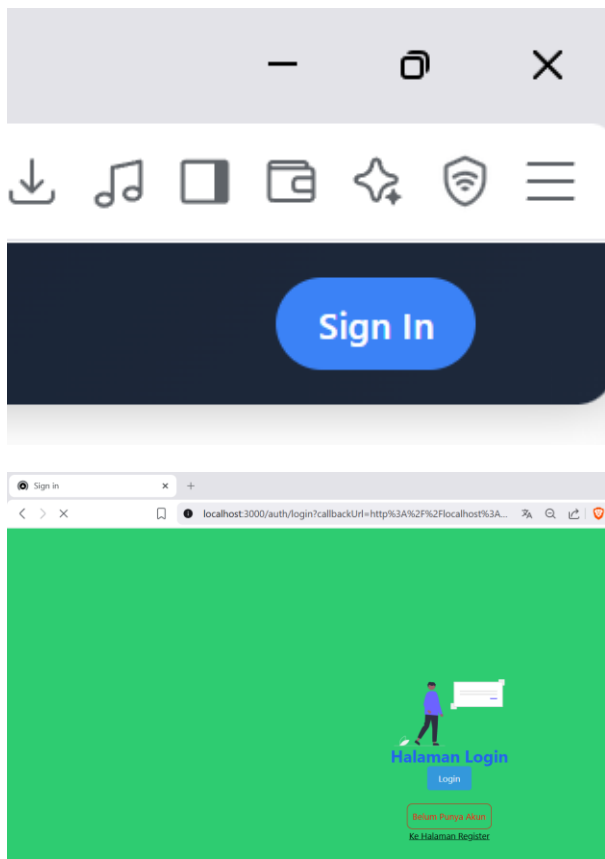
**BAGIAN 1 – Custom Login Page**

Tambahkan custom page di NextAuth line 55-57

```
pemograman_framework - [...nextauth].ts

54 pages: {
55 signIn: "/auth/login",
56 },
```

Jalankan browser <http://localhost:3000/> dan klik sign in maka akan diarahkan ke login



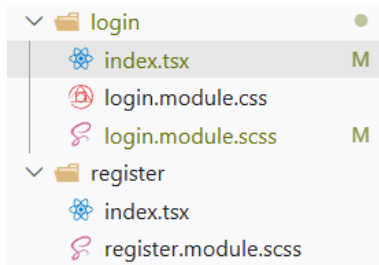
Benar

**BAGIAN 2 – Handle Login di Frontend**

Copy paste isi dari register/index.tsx ke file login/index.tsx

Copy paste isi dari register/register.module.scss ke file login/login.module.scss

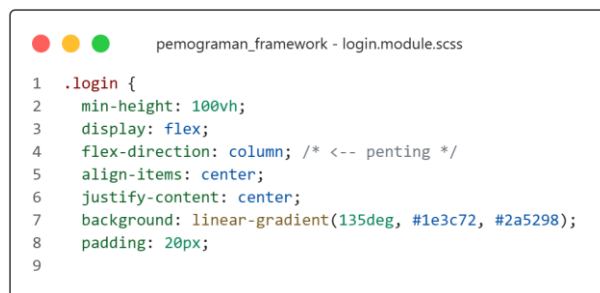
Semua text register pada file index.tsx pada folder login diubah menjadi login



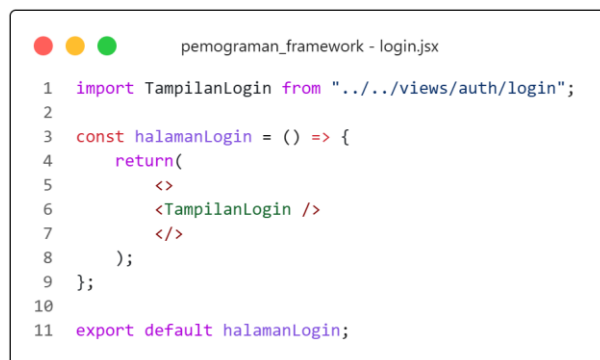
Jangan lupa setting link hrefnya



Lakukan hal yang sama pada file login.module.scss rubah text register menjadi login



Cek pada file login.tsx pada pages/auth



Jalankan browser localhost:3000/auth/login. Tampilannya akan sama dengan register

## Halaman Login

Email

Full Name

Password

Login

Sudah punya akun? [Ke Halaman Register](#)

Pada tampilan login kita tidak perlu hapus fullname jadi pada folder views/auth/login/index.tsx hapus fullname

Sehingga hasilnya seperti berikut :

## Halaman Login

Email

Password

Login

Sudah punya akun? [Ke Halaman Register](#)

Buka file index.tsx pada folder views/auth/login dan modifikasi codenya seperti berikut ( Untuk line 64 sampai kebawah tidak ada perubahan )

```
pemograman_framework - index.tsx

51 // }
52 try {
53 const res = await signIn("credentials", {
54 redirect: false,
55 email: event.target.email.value,
56 password: event.target.password.value,
57 callbackUrl,
58 });
59
60 // console.log("SignIn response:", res);
61 if (!res?.error) {
62 setIsLoading(false);
63 push("/");
64 } else {
65 setIsLoading(false);
66 setError(res?.error || "Login failed");
67 }
68 } catch (error) {
69 setIsLoading(false);
70 setError("Wrong email or password");
71 }
```

Note pastikan tulisan password pada event.password.value pada line 48 sama dengan yang ada di

```
pemograman_framework - index.tsx

104 <input
105 type="password"
106 id="password"
107 name="password"
108 placeholder="Password"
109 className={style.login__form__item__input}
110 />
```

Buka file servicefirebase.ts dan tambahkan code di line 25-38

```
pemograman_framework - servicefirebase.ts

26 export async function signIn(email: string) {
27 const q = query(collection(db, "users"), where("email", "=", email));
28 const querySnapshot = await getDocs(q);
29 const data = querySnapshot.docs.map((doc) => ({
30 id: doc.id,
31 ...doc.data(),
32 }));
33 if(data){
34 return data[0];
35 } else {
36 return null;
37 }
38 }
```

### BAGIAN 3 – Authorize di NextAuth (Database Login)

Buka file [...nextauth].ts modifikasi menjadi berikut ( pada bagian providers )

```

pemograman_framework - [...nextauth].ts
1 import NextAuth, { NextAuthOptions } from "next-auth";
2 import CredentialsProvider from "next-auth/providers/credentials";
3 import { signIn } from "@/utils/db/servicefirebase";
4 import bcrypt from "bcrypt";
5
6 export const authOptions: NextAuthOptions = {
7 session: {
8 strategy: "jwt",
9 },
10 secret: process.env.NEXTAUTH_SECRET,
11 providers: [
12 CredentialsProvider({
13 name: "Credentials",
14 credentials: {
15 // fullname: { label: "Full Name", type: "text" },
16 email: { label: "Email", type: "email" },
17 password: { label: "Password", type: "password" },
18 },
19 async authorize(credentials) {
20 if (!credentials?.email || !credentials?.password) return null;
21
22 const user: any = await signIn(credentials.email);
23 if (user) {
24 const isPasswordValid = await bcrypt.compare(
25 credentials.password,
26 user.password,
27);
28 if (isPasswordValid) {
29 // Pastikan mengembalikan objek user yang bersih
30 return {
31 id: user.id,
32 email: user.email,
33 fullname: user.fullname,
34 role: user.role,
35 };
36 }
37 }
38 return null;
39 },
40 }),
41],

```

## BAGIAN 4 – Tambahkan Role ke Token

JWT Callback pada file [...nextauth].ts Modifikasi menjadi

```

pemograman_framework - [...nextauth].ts
43 callbacks: {
44 async jwt({ token, account, profile, user }: any) {
45 if (account?.provider === "credentials" && user) {
46 token.email = user.email;
47 token.fullname = user.fullname;
48 token.role = user.role;
49 }
50 // console.log("jwt callback", {token, account, profile, user});
51 return token;
52 },
53 async session({ session, token }: any) {
54 if (token.email) {
55 session.user.email = token.email;
56 }
57 if (token.fullname) {
58 session.user.fullname = token.fullname;
59 }
60 if (token.role) {
61 session.user.role = token.role;
62 }
63 // console.log("session callback", {session, token});
64 return session;
65 },
66 },

```

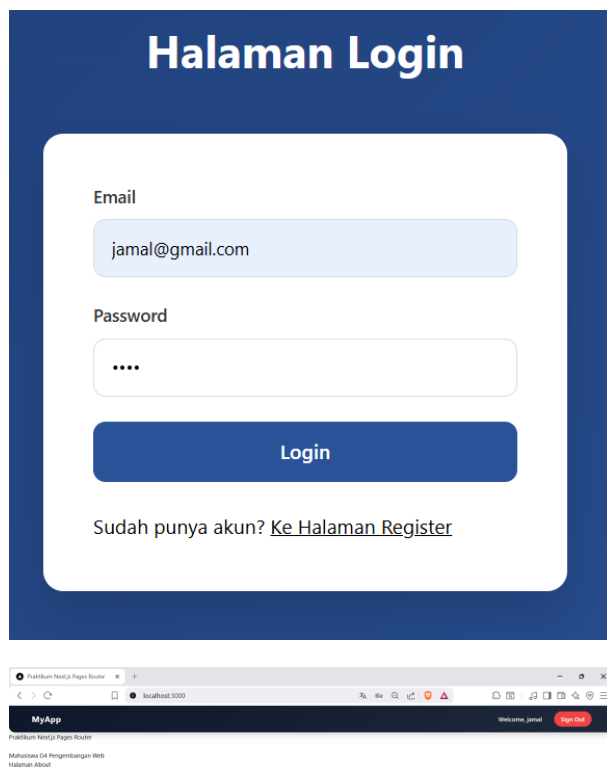
Perbaiki kode pada views/auth/login/index.tsx

```

pemograman_framework - index.tsx
16 const formData = new FormData(event.currentTarget);
17 const email = formData.get("email") as string;
18 const password = formData.get("password") as string;
19 try {
20 const res = await signIn("credentials", {
21 redirect: false,
22 email,
23 password,
24 callbackUrl,
25 });

```

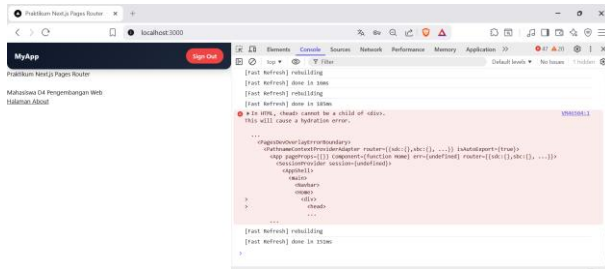
Jalankan browser <http://localhost:3000/auth/login>



Note ERROR :

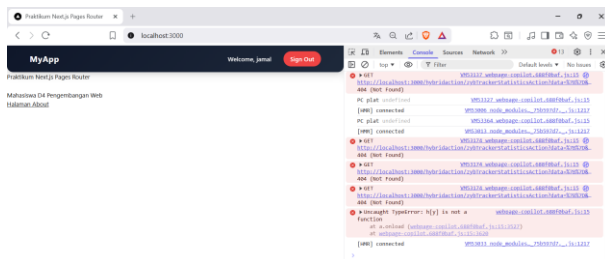
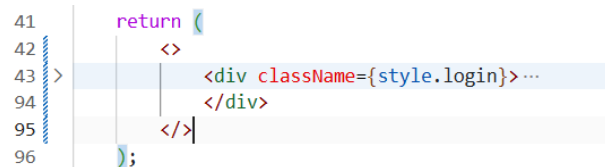
JIKA TERDAPAT ERROR SEPERTI INI DISEBABKAN KARENA

In this case, the problem is that a `<head>` tag is being rendered inside a `<div>`, which is invalid HTML. The `<head>` element must be a direct child of `<html>`, not nested inside other elements.



SOLUSINYA BUKA FILE SRC/VIEWS/AUTH/LOGIN/INDEX.TSX

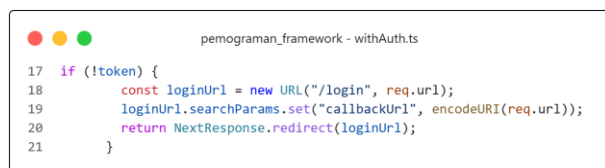
TAMBAHKAN `<>` `</>` SEPERTI PADA GAMBAR BERIKUT ( LINE 42 DAN 96)



**Note:** Error 404 berasal dari Extension browser (seperti Baidu, IDM, atau tracker lainnya) atau script pihak ketiga yang mencoba mengirim data statistik/tracking.

## BAGIAN 5 – Callback URL Logic

Modifikasi withAuth.ts pada folder src/middleware



Tujuannya:

→Setelah login, user kembali ke halaman sebelumnya.

## BAGIAN 6 – Membuat halaman Admin dan authoriz

Buat halaman admin

Pages/admin/index.tsx

Pada index.tsx tambahkan code berikut



```
pemograman_framework - index.tsx
1 const HalamanAdmin = () => {
2 return (
3 <div className="admin">
4 <h1>Halaman Admin</h1>
5 <p>
6 Selamat datang di halaman admin! Anda memiliki akses penuh ke semua
7 fitur dan data di aplikasi ini. Di sini, Anda dapat mengelola
8 pengguna, melihat laporan, dan melakukan tugas administratif lainnya.
9 Pastikan untuk menggunakan hak akses Anda dengan bijak dan menjaga
10 keamanan data pengguna.
11 </p>
12 </div>
13);
14 };
15
16 export default HalamanAdmin;
```

## Modifikasi withAuth.ts

```
pemograman_framework - withAuth.ts
1 import { getToken } from "next-auth/jwt";
2 import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";
3
4 const hanyaAdmin = ["/admin"];
5
6 export default function withAuth(
7 middleware: NextMiddleware,
8 requireAuth: string[] = [],
9) {
10 return async (req: NextRequest, next: NextFetchEvent) => {
11 const pathname = req.nextUrl.pathname;
12
13 if (requireAuth.includes(pathname)) {
14 const token = await getToken({
15 req,
16 secret: process.env.NEXTAUTH_SECRET,
17 });
18
19 if (!token) {
20 const loginUrl = new URL("/login", req.url);
21 loginUrl.searchParams.set("callbackUrl", encodeURIComponent(req.url));
22 return NextResponse.redirect(loginUrl);
23 }
24 if (token.role !== "admin" && hanyaAdmin.includes(pathname)){
25 return NextResponse.redirect(new URL("/", req.url));
26 }
27 }
28 return middleware(req, next);
29 }
30 }
```

## Perbaikan pada middleware.ts

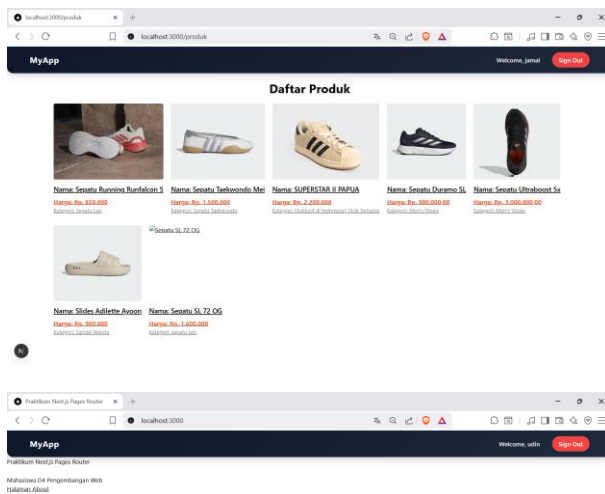
```

pemograman_framework - middleware.ts

1 import { NextResponse } from "next/server";
2 import type { NextRequest } from "next/server";
3 import withAuth from "../middleware/withAuth";
4
5 const baseMiddleware = (req: NextRequest) => {
6 return NextResponse.next();
7 };
8
9 export default withAuth(baseMiddleware, [
10 "/produk",
11 "/about",
12 "/lapar",
13 "/profil",
14 "/admin",
15]);
16
17 export const config = {
18 matcher: [
19 "/produk",
20 "/about",
21 "/lapar",
22 "/profil",
23 "/admin/",
24],
25 };

```

Jalankan browser localhost:3000/produk dan pada status sudah login. Rubah urlnya menjadi <http://localhost:3000/admin> maka user akan diarahkan ke localhost. Pada intinya role selain admin tidak bisa mengakses



Untuk mencoba halaman admin rubah role pada firebase pada salah satu akun dan jalankan <http://localhost:3000/admin>

```
email: "udin@gmail.com"
fullname: "udin"
password: "$2b$10$.A5pybz2fVfMxAuR36ALtO2jKKUmS/qHzUdW0SjUJCDzKq4wsu6."
role: "admin"
```



## D. PENGUJIAN

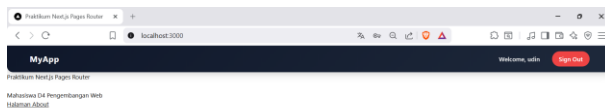
### Uji 1 – Login Valid

Input:

- Email benar
- Password benar

Hasil:

- Login berhasil



- Redirect sesuai callbackUrl

```
pemograman_framework - index.tsx
28 if (!res?.error) {
29 setIsLoading(false);
30 push("/");
}
```

Benar, akan diarahkan ke localhost:3000

## Uji 2 – Password Salah

Input:

- Email benar
- Password salah

**Halaman Login**

Email  
udin@gmail.com

Password  
.....

Login

Sudah punya akun? [Ke Halaman Register](#)

Hasil:

- Error message tampil

Credentials: Signin

## Halaman Login

Email

Password

Login

Sudah punya akun? [Ke Halaman Register](#)

- Tidak login

Benar, hanya tampil teks pesan error

### Uji 3 – Akses Admin sebagai User

Login sebagai:

- role: user

## Halaman Login

Email

Password

Login

Sudah punya akun? [Ke Halaman Register](#)



Akses:

/admin

Hasil:

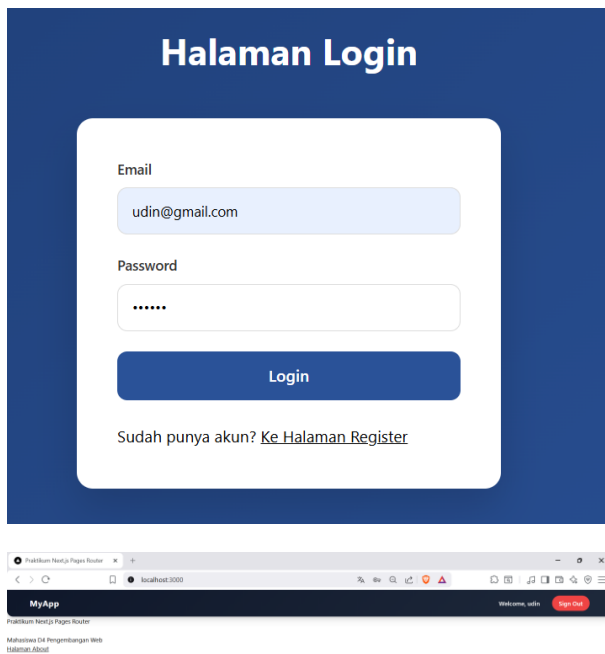
- Redirect ke home

Benar

#### Uji 4 – Akses Admin sebagai Admin

Login sebagai:

- role: admin



Akses:

/admin

Hasil:

- Bisa masuk halaman admin



Benar

## E. STRUKTUR DATABASE USERS

Collection: users

| Field    | Tipe            |
|----------|-----------------|
| email    | string          |
| password | string (hashed) |
| role     | string          |
| fullName | string          |

## F. PERBANDINGAN SISTEM

| Fitur      | Sebelum   | Sekarang     |
|------------|-----------|--------------|
| Login      | Hardcoded | Database     |
| Password   | Plaintext | Hashed       |
| Role       | Tidak ada | Ada          |
| Redirect   | Manual    | Callback URL |
| Middleware | Basic     | Role-based   |

## G. TUGAS PRAKTIKUM

### 1. Implementasikan login database.

Sudah

**Halaman Login**

Email  
udin@gmail.com

Password  
\*\*\*\*\*

Login

Sudah punya akun? [Ke Halaman Register](#)

## 2. Tambahkan role pada user.

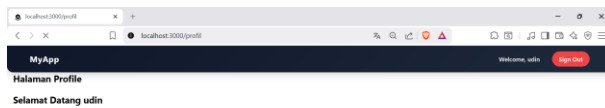
Sudah

```
pemograman_framework - servicefirebase.ts

56 userData: {
57 email: string;
58 fullname: string;
59 password: string;
60 role?: string;
61 },
```

## 3. Buat halaman:

o /profile



o /admin



## 4. Proteksi /admin hanya untuk admin.

```
pemograman_framework - withAuth.ts

4 const hanyaAdmin = ["/admin"];
```

```
pemograman_framework - withAuth.ts

27 if (token.role !== "admin" && hanyaAdmin.includes(pathname)){
28 return NextResponse.redirect(new URL("/", req.url));
29 }
```



```

pemograman_framework - middleware.ts

1 import { NextResponse } from "next/server";
2 import type { NextRequest } from "next/server";
3 import withAuth from "../middleware/withAuth";
4
5 const baseMiddleware = (req: NextRequest) => {
6 return NextResponse.next();
7 };
8
9 export default withAuth(baseMiddleware, [
10 "/produk",
11 "/about",
12 "/lapar",
13 "/profil",
14 "/admin",
15]);
16
17 export const config = {
18 matcher: [
19 "/produk",
20 "/about",
21 "/lapar",
22 "/profil",
23 "/admin/",
24],
25 };

```

## 5. Implementasikan callback URL.

Sudah

```

pemograman_framework - withAuth.ts

17 if (!token) {
18 const loginUrl = new URL("/login", req.url);
19 loginUrl.searchParams.set("callbackUrl", encodeURIComponent(req.url));
20 return NextResponse.redirect(loginUrl);
21 }

```

## H. PERTANYAAN ANALISIS

1. Mengapa password harus diverifikasi dengan bcrypt.compare?

Untuk memastikan kecocokan hash password dengan yang tersimpan secara aman tanpa menyimpan password asli.

2. Mengapa role disimpan di token?

Agar informasi akses pengguna dapat diakses dengan cepat dan aman di sisi client maupun server tanpa query database berulang.

3. Apa fungsi callbackUrl?

CallbackUrl berfungsi mengarahkan pengguna kembali ke halaman tujuan setelah proses login berhasil.

4. Mengapa middleware penting untuk security?

Middleware penting untuk security karena dapat memproteksi akses route sebelum konten atau fungsi dijalankan.

5. Apa risiko jika role tidak dicek di middleware?

Pengguna tanpa izin dapat mengakses halaman atau fungsi yang seharusnya terbatas, menyebabkan kebocoran data atau tindakan tidak sah.

## **I. OUTPUT YANG DIHARAPKAN**

Mahasiswa menghasilkan:

- ✓ Login terhubung database
- ✓ Password diverifikasi
- ✓ Custom login page
- ✓ Redirect sesuai callback
- ✓ URL Middleware aktif
- ✓ Role-based access berjalan

## **J. INSIGHT PENTING**

Level ini sudah masuk kategori:

Production-ready authentication system

Yang sudah mencakup:

- Authentication
- Authorization
- Session management
- Role-based access control

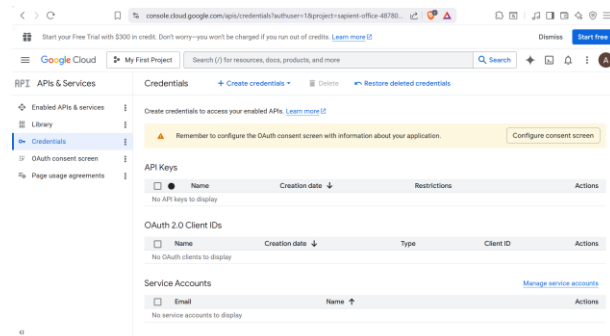
# LAPORAN JOBSHEET 17: IMPLEMENTASI LOGIN GOOGLE PROVIDER DENGAN NEXTAUTH.JS + FIREBASE MATKUL PEMOGGRAMAN BERBASIS NETWORK

AHMAD DZUL FADHLI HANNAN | TI3F | 2341720106

## B. KONFIGURASI GOOGLE OAUTH

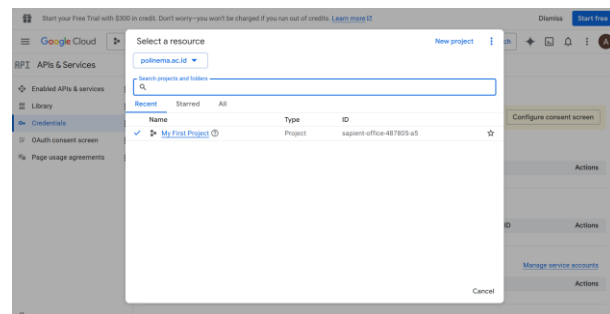
Langkah 1 – Masuk ke Google Cloud Console Buka:

o <https://console.cloud.google.com/apis/credentials>



Langkah 2 – Buat Project Baru

- Klik New Project



Nama project: MyAppNext

You have 10 projects remaining in your quota. Request an increase or delete projects. [Learn more](#)

[Manage Quotas](#)

Project name \*  
MyAppNext

Project ID: myappnext-492814. It cannot be changed later. [Edit](#)

Organization \*  
polinema.ac.id

Select an organization to attach it to a project. This selection can't be changed later.

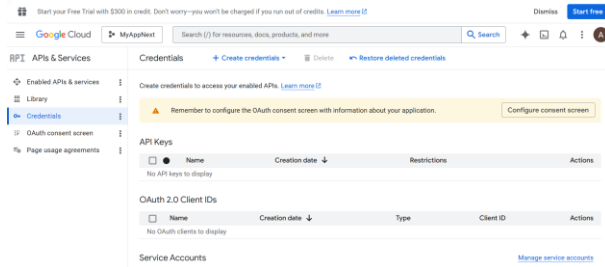
Parent resource \*  
polinema.ac.id [Browse](#)

Select the organization or folder where this resource will be created

[Create](#) [Cancel](#)

Klik Create

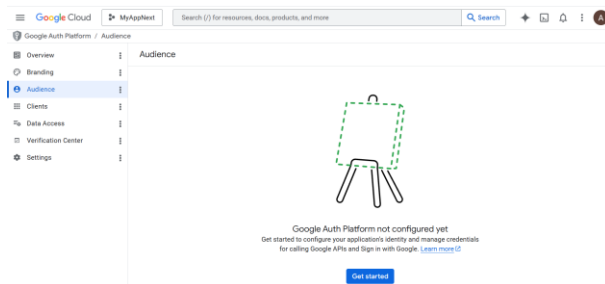
Setelah berhasil klik <https://console.cloud.google.com/apis/credentials> pastikan projectnya MyAppNext



### Langkah 3 – Konfigurasi OAuth Consent Screen

1. Pilih OAuth consent screen

2. Pilih Get Started



3. Maka akan muncul seperti berikut dan isikan

1 App Information

App name \*  
my-app

The name of the app asking for consent

For users to contact you with questions about their consent. [Learn more](#)

Next

✓ App Information

2 Audience

☐ Internal ?

Only available to users within your organization. You will not need to submit your app for verification. [Learn more about user type](#)

☒ External ?

Available to any test user with a Google Account. Your app will start in testing mode and will only be available to users you add to the list of test users. Once your app is ready to push to production, you may need to verify your app. [Learn more about user type](#)

Next

3 Contact Information

✓ App Information

✓ Audience

3 Contact Information

These email addresses will be used by Google to notify you about any changes to your project.

Next

4 Finish

Create Cancel

✓ App Information

✓ Audience

✓ Contact Information

4 Finish

☒ I agree to the [Google API Services: User Data Policy](#)

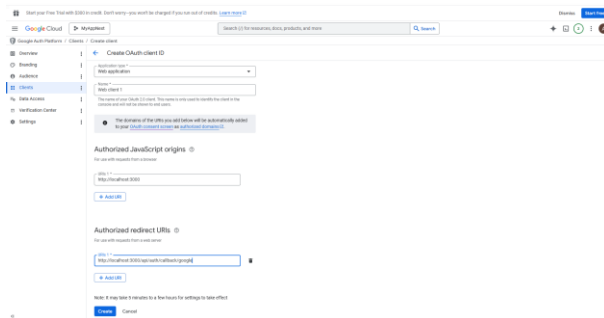
Continue

Create Cancel

Klik create

## Langkah 4 – Buat OAuth Credentials

### 1. Klik create client pada Clients



## C. TAMBAHKAN ENVIRONMENT VARIABLES

### 1. Copy dan paste client ID dan Client secret ke .env

```
pemograman_framework - client_se

1 {
2 "web": {
3 "client_id": "8902677789",
4 "project_id": "myappnext",
5 "auth_uri": "https://accounts.google.com/o/oauth2/auth",
6 "token_uri": "https://accounts.google.com/o/oauth2/token",
7 "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
8 "client_secret": "GOCSPX",
9 "redirect_uris": [
10 "http://localhost:3000"
11],
12 "javascript_origins": [
13 "http://localhost:3000"
14]
15 }
16 }
```

```
1 FIREBASE_API_KEY=AizaSyB6
2 FIREBASE_AUTH_DOMAIN=fram
3 FIREBASE_PROJECT_ID=frame
4 FIREBASE_STORAGE_BUCKET=f
5 FIREBASE_MESSAGING_SENDER
6 FIREBASE_APP_ID=1:2357282
7
8 REVALIDATE_TOKEN=12345678
9
10 // gunakan website https:
11 // lalu masukkan kedalam
12 NEXTAUTH_SECRET=94ddd5597
13 GOOGLE_CLIENT_ID=89026777
14 GOOGLE_CLIENT_SECRET=GOCSPX
```

## D. KONFIGURASI GOOGLE PROVIDER DI NEXTAUTH DAN HANDLE CALLBACK JWT & SESSION

1. Buka file [...nextauth].ts pada folder api/auth dan modifikasi menjadi berikut

```
pemograman_framework - [...nextauth].ts
5 import GoogleProvider from "next-auth/providers/google";
```

```
pemograman_framework - [...nextauth].ts
43 GoogleProvider({
44 clientId: process.env.GOOGLE_CLIENT_ID || "",
45 clientSecret: process.env.GOOGLE_CLIENT_SECRET || "",
46 }),
47],
```

```
pemograman_framework - [...nextauth].ts
57 // Jika login dengan Google, tambahkan informasi yang diperlukan ke token
58 if (account?.provider === "google") {
59 const data = {
60 fullname: user?.name,
61 email: user?.email,
62 image: user?.image,
63 type: account.provider,
64 }
65 // console.log("Google login data:", { data });
66 token.fullname = data.fullname;
67 token.email = data.email;
68 token.image = data.image;
69 token.type = data.type;
70 }
```

```
pemograman_framework - [...nextauth].ts
80 if (token.image) {
81 session.user.image = token.image;
82 }
```

```
pemograman_framework - [...nextauth].ts
86 if(token.type) {
87 session.user.type = token.type;
88 }
```

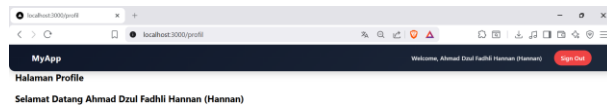
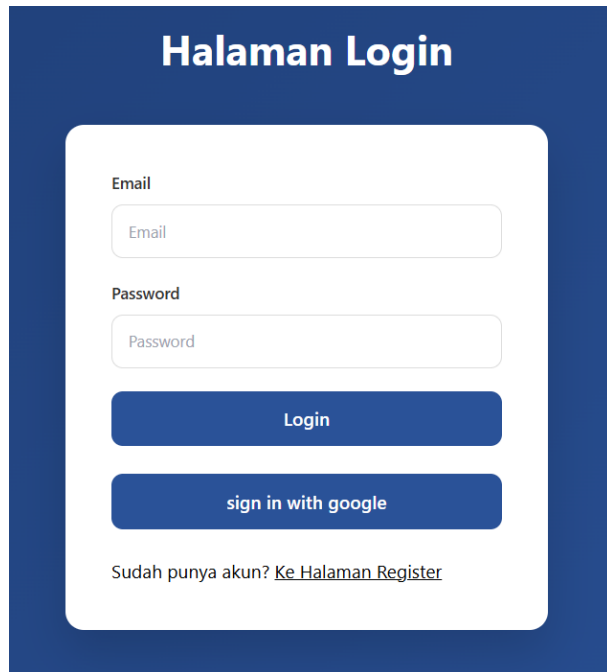
## E. TAMBAHKAN BUTTON LOGIN GOOGLE

1. Modifikasi file index.tsx pada folder views/auth/login

```
pemograman_framework - index.tsx
89

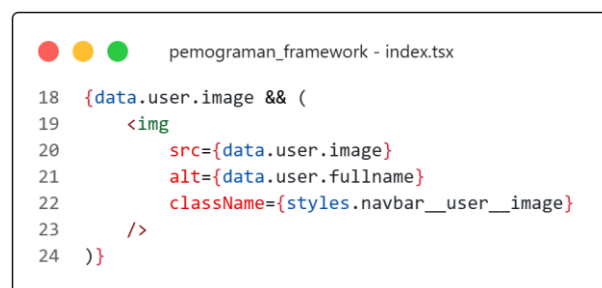
90 /* Button Login Google */
91 <button
92 onClick={() => signIn("google", { callbackUrl, redirect: false })}
93 className={style.login_form_item_button}
94 disabled={isLoading}
95 >
96 {isLoading ? "Loading..." : "sign in with google"}
97 </button>
```

2. Jalankan browser localhost:3000/auth/login masuk melalui sign in with google. Jika berhasil maka akan terhubung dengan akun google.



3. Menampilkan image dari google

o Buka file index.tsx dan tambahkan code berikut



Buka file navbar.module.css dan tambahkan code berikut



```

71 /* Responsive */
72 @media (max-width: 768px) {
73 .navbar {
74 padding: 0 20px;
75 }
76
77 .navbar__brand {
78 font-size: 18px;
79 }
80
81 .navbar__user {
82 display: none;
83 }
84 }
85
86 .navbar__user {
87 display: flex;
88 align-items: center;
89 gap: 12px;
90 font-size: 16px;
91 font-weight: 500;
92 opacity: 0.95;
93 }
94 /* Avatar Image */
95 .navbar__user__avatar {
96 width: 42px;
97 height: 42px;
98 border-radius: 50%;
99 object-fit: cover;
100 border: 2px solid #3b82f6;
101 box-shadow: 0 0 10px rgba(59, 130, 246, 0.4);
102 transition: all 0.3s ease;
103 }
104
105 .navbar__user__image:hover {
106 transform: scale(1.1);
107 box-shadow: 0 0 15px rgba(59, 130, 246, 0.8);
108 }
109

```

Jika berhasil maka tampilannya akan seperti berikut



## G. SIMPAN DATA GOOGLE KE DATABASE

Buka file servicefirebase.ts pada folder src/utils/db/ dan tambahkan beberapa kode berikut

● ● ● pemograman\_framework - servicefirebase.ts

```
1 import {
2 getFirestore,
3 collection,
4 getDocs,
5 Firestore,
6 getDoc,
7 doc,
8 query,
9 addDoc,
10 where,
11 updateDoc,
12 } from "firebase/firestore";
```

Tambahkan juga code berikut

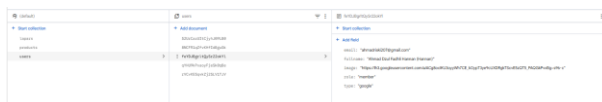
● ● ● pemograman\_framework - servicefirebase.ts

```
103 export async function signInWithGoogle(userData: any, callback: any) {
104 try {
105 const q = query(
106 collection(db, "users"),
107 where("email", "==", userData.email),
108);
109
110 const querySnapshot = await getDocs(q);
111 const data: any = querySnapshot.docs.map((doc) => ({
112 id: doc.id,
113 ...doc.data(),
114 }));
115
116 if (data.length > 0) {
117 // User sudah ada, update data
118 userData.role = data[0].role;
119 await updateDoc(doc(db, "users", data[0].id), userData);
120 callback({
121 status: true,
122 message: "User registered and logged in with Google",
123 data: userData,
124 });
125 } else {
126 // User belum ada, buat data baru
127 userData.role = "member";
128 await addDoc(collection(db, "users"), userData);
129 callback({
130 status: true,
131 message: "User registered and logged in with Google",
132 data: userData,
133 });
134 }
135 } catch (error: any) {
136 // Tangani error di sini
137 callback({
138 status: false,
139 message: "Failed to register user with Google",
140 });
141 }
142 }
143
```

Panggil Service di JWT Callback buka file [...nextAuth].ts

```
pemograman_framework - [...nextauth].ts
57 // Jika login dengan Google, tambahkan informasi yang diperlukan ke token
58 if (account?.provider === "google") {
59 const data = {
60 fullname: user.name,
61 email: user.email,
62 image: user.image,
63 type: account.provider,
64 };
65
66 await signInWithGoogle(data, (result: any) => {
67 // Pastikan mengecek result.status sesuai dengan object yang dikirim
68 if (result.status) {
69 token.fullname = result.data.fullname;
70 token.email = result.data.email;
71 token.image = result.data.image;
72 token.type = result.data.type;
73 token.role = result.data.role;
74 }
75 });
76 }
77 return token;
```

Jalankan browser dan login menggunakan akun google setelah cek di firebase, jika data akun googlenya masuk ke database maka anda telah berhasil



Benar

## H. PENGUJIAN

### 1. Login Google pertama kali

Hasil:

Data tersimpan di Firestore

```
email: "ahmadrizki207@gmail.com"
fullname: "Ahmad Dzul Fadli Hannan (Hannan)"
image: "https://lh3.googleusercontent.com/a/ACg8ocIKU3oyyWh7CE_kOypT3ye9cUXDRgkTSvIESzQT5_PAQgKPxvBg=s96-c"
role: "member"
type: "google"
```

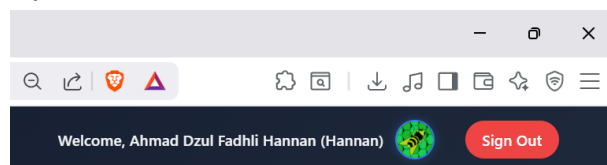
Benar

### 2. Login Google kedua kali

Hasil:

Data diupdate

Update Foto Profil



Pada firebase

Sebelum

```
email: "ahmadriski207@gmail.com"
fullName: "Ahmad Dzul Fadhlil Hannan (Hannan)"
image: "https://lh3.googleusercontent.com/a/ACg8oclpKU3oyyWhTCE_kOypT3ye9cUXDRgkTScvE5zOT5_PAQgKPxvBg-s96-c"
role: "member"
type: "google"
```

## Sesudah

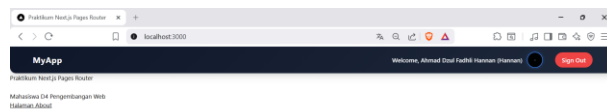
```
email: "ahmadriski207@gmail.com"
fullName: "Ahmad Dzul Fadhlil Hannan (Hannan)"
image: "https://lh3.googleusercontent.com/a/ACg8oclpMG2-TBXcd1Mwu0I0IM5djpH5rvSthmoxclwd8nUKTTXGaV3-s96-c"
role: "member"
type: "google"
```

## Benar

### 3. User role member akses /admin

#### Hasil:

#### Redirect /



## Benar

### 4. User role admin akses /admin

#### Hasil:

#### Bisa masuk

#### Ubah role jadi admin

```
email: "ahmadriski207@gmail.com"
fullName: "Ahmad Dzul Fadhlil Hannan (Hannan)"
image: "https://lh3.googleusercontent.com/a/ACg8oclpMG2-TBXcd1Mwu0I0IM5djpH5rvSthmoxclwd8nUKTTXGaV3-s96-c"
role: "admin"
type: "google"
```

## Relog

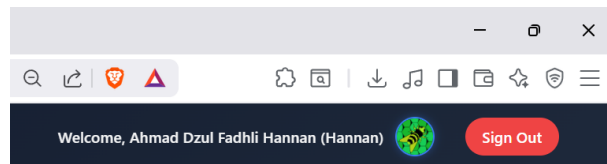
#### Akses /admin



## Benar

### 5. Avatar tampil

#### Hasil:



## Ya

## ANALISIS & DISKUSI

### 1. Apa perbedaan login credential dan login Google?

Login credential menggunakan email dan password yang diverifikasi sendiri, sedangkan login Google menggunakan autentikasi pihak ketiga tanpa perlu menyimpan password.

2. Mengapa data Google tetap perlu disimpan ke database?

Agar aplikasi dapat mengelola informasi tambahan seperti role dan data pengguna lainnya.

3. Apa fungsi JWT callback?

JWT callback berfungsi untuk menyimpan dan memodifikasi data user ke dalam token agar bisa digunakan di seluruh aplikasi.

4. Mengapa perlu multi-role?

Untuk membedakan hak akses pengguna sesuai dengan peran masing-masing.

5. Apa risiko jika tidak menyimpan user ke database?

Tidak bisa mengontrol akses, menyimpan data tambahan, atau mengelola pengguna secara efektif.

## TUGAS MANDIRI

### 1. Tambahkan role editor

withAuth.ts, hanya editor dan admin yang bisa akses path /editor

```
pemograman_framework - withAuth.ts
1 import { getToken } from "next-auth/jwt";
2 import {
3 NextFetchEvent,
4 NextMiddleware,
5 NextRequest,
6 NextResponse,
7 } from "next/server";
8
9 const hanyaAdmin = ["/admin"];
10 const hanyaEditor = ["/editor"];
11
12 export default function withAuth(
13 middleware: NextMiddleware,
14 requireAuth: string[] = [],
15) {
16 return async (req: NextRequest, next: NextFetchEvent) => {
17 const pathname = req.nextUrl.pathname;
18 // console.log("PATH:", pathname);
19 const isProtected = requireAuth.some((path) => pathname.startsWith(path));
20
21 const isAdminRoute = hanyaAdmin.some((path) => pathname.startsWith(path));
22
23 const isEditorRoute = hanyaEditor.some((path) => pathname.startsWith(path));
24
25 if (isProtected) {
26 const token = await getToken({
27 req,
28 secret: process.env.NEXTAUTH_SECRET,
29 });
30
31 if (!token) {
32 const loginUrl = new URL("/auth/login", req.url);
33 loginUrl.searchParams.set("callbackUrl", encodeURI(req.url));
34 return NextResponse.redirect(loginUrl);
35 }
36 if (isAdminRoute && token.role !== "admin") {
37 return NextResponse.redirect(new URL("/", req.url));
38 }
39 if (isEditorRoute && !["admin", "editor"].includes(token.role as string)) {
40 return NextResponse.redirect(new URL("/", req.url));
41 }
42 }
43 return middleware(req, next);
44 };
45 }
46
```

## Middleware.ts

```
pemograman_framework - middleware.ts

9 export default withAuth(baseMiddleware, [
10 "/produk",
11 "/about",
12 "/lapar",
13 "/profil",
14 "/admin",
15 "/editor",
16]);
17
18 export const config = {
19 matcher: [
20 "/produk",
21 "/about",
22 "/lapar",
23 "/profil",
24 "/admin:path*",
25 "/editor:path*",
26],
```

## 2. Buat halaman khusus editor

```
pemograman_framework - index.tsx

1 const HalamanEditor = () => {
2 return (
3 <div className="editor">
4 <h1>Halaman Editor</h1>
5 <p>
6 Halaman ini khusus untuk role editor dan admin. Gunakan halaman ini untuk
7 melakukan pengelolaan konten aplikasi.
8 </p>
9 </div>
10);
11 };
12
13 export default HalamanEditor;
14
```

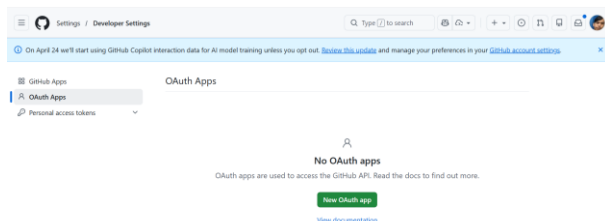
## Hasil



## 3. Tambahkan provider GitHub

### Langkah 1 – Konfigurasi di Github

Buka: <https://github.com/settings/developers>



Klik "New OAuth App"

Isi:

Application name: MyAppNext

Homepage URL: <http://localhost:3000>

Authorization callback URL: <http://localhost:3000/api/auth/callback/github>

#### Register a new OAuth app

Application name \*

MyAppNext

Something users will recognize and trust.

Homepage URL \*

<http://localhost:3000>

The full URL to your application homepage.

Application description

Application description is optional

This is displayed to all users of your application.

Authorization callback URL \*

<http://localhost:3000/api/auth/callback/github>

Your application's callback URL. Read our [OAuth documentation](#) for more information.

☐ Enable Device Flow

Allow this OAuth App to authorize users via the Device Flow.

Read the [Device Flow documentation](#) for more information.

Register application

Cancel

Klik Register application

## Langkah 2 – Tambahkan Environment Variables

Tambahkan Client ID dan Client Secret pada env.local

pemograman\_framework - .env.local

15 GITHUB\_ID=

16 GITHUB\_SECRET=

Modifikasi [...nextauth].ts

## Langkah 3 - Konfigurasi google provider di nextauth dan handle callback jwt & session

Modifikasi [...nextAuth].ts

pemograman\_framework - [...nextauth].ts

7 import GitHubProvider from "next-auth/providers/github";

```

13 providers: [
14 > CredentialsProvider({ ...
42 })),
43 > GoogleProvider({ ...
46 })),
47 GitHubProvider({
48 clientId: process.env.GITHUB_ID || "",
49 clientSecret: process.env.GITHUB_SECRET || "",
50 issuer: "https://github.com/login/oauth",
51 })),
52],

```

## Modifikasi views/auth/login/index.ts

```

pemograman_framework - index.tsx
98 /* Button Login Github */
99 <button
100 onClick={() => signIn("github", { callbackUrl, redirect: false })}
101 className={style.login_form_item_button}
102 disabled={isLoading}
103 >
104 {isLoading ? "Loading..." : "sign in with github"}
105 </button>

```

## Tambahkan fungsi signInWithGithub pada servicefirebase.ts

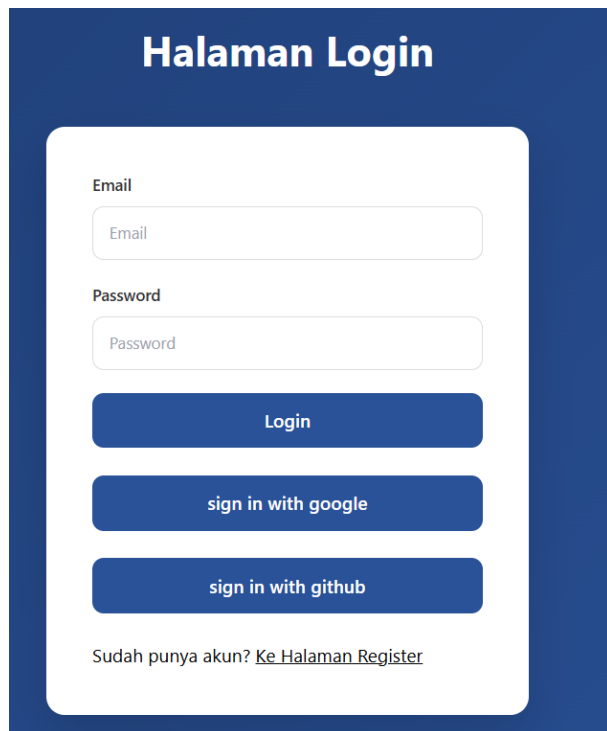
```

pemograman_framework - servicefirebase.ts
143 export async function signInWithGithub(userData: any, callback: any) {
144 try {
145 const q = query(
146 collection(db, "users"),
147 where("email", "==", userData.email),
148);
149
150 const querySnapshot = await getDocs(q);
151 const data: any = querySnapshot.docs.map((doc) => ({
152 id: doc.id,
153 ...doc.data(),
154 }));
155 if (data.length > 0) {
156 // User sudah ada, update data
157 userData.role = data[0].role;
158 await updateDoc(doc(db, "users", data[0].id), userData);
159 callback({
160 status: true,
161 message: "User registered and logged in with Github",
162 data: userData,
163 });
164 } else {
165 // User belum ada, buat data baru
166 userData.role = "member";
167 await addDoc(collection(db, "users"), userData);
168 callback({
169 status: true,
170 message: "User registered and logged in with Github",
171 data: userData,
172 });
173 }
174 } catch (error: any) {
175 // Tangani error di sini
176 callback({
177 status: false,
178 message: "Failed to register user with Github",
179 });
180 }
181 }
182

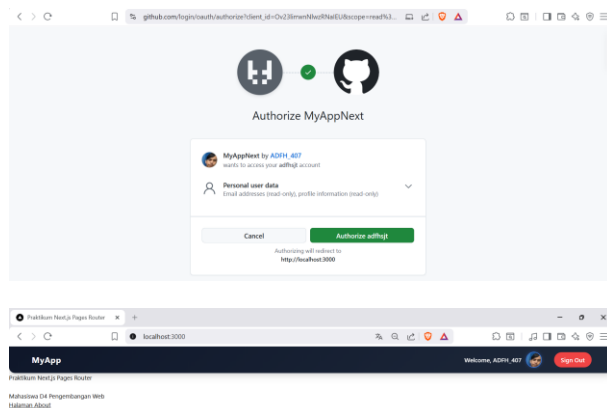
```



## Langkah 4 – Pengujian



Masuk dengan akun github



Bisa

## 4. Refactor service agar reusable

Hapus fungsi `signInWithGoogle` dan `signInWithGithub` pada `servicefirebase.ts`, dan Ganti dengan kode reusable berikut:

```

102 export async function signInWithOAuth(provider: string, userData: any, callback: any) {
103 try {
104 const q = query(
105 collection(db, "users"),
106 where("email", "==", userData.email),
107);
108 const querySnapshot = await getDocs(q);
109 const data: any = querySnapshot.docs.map((doc) => ({
110 id: doc.id,
111 ...doc.data(),
112 }));
113 if (data.length > 0) {
114 // User sudah ada, update data
115 userData.role = data[0].role;
116 await updateDoc(doc(db, "users", data[0].id), userData);
117 callback({
118 status: true,
119 message: 'User registered and logged in with ${provider}',
120 data: userData,
121 });
122 } else {
123 // User belum ada, buat data baru
124 userData.role = "member";
125 await addDoc(collection(db, "users"), userData);
126 callback({
127 status: true,
128 message: 'User registered and logged in with ${provider}',
129 data: userData,
130 });
131 }
132 } catch (error: any) {
133 // Tangani error di sini
134 callback({
135 status: false,
136 message: 'Failed to register user with ${provider}',
137 });
138 }
139 }
140

```

Sekarang kode jadi reusable dan tidak duplikat kode lagi

## 5. Gunakan next/image untuk optimasi avatar

Pada navbar/index.tsx

```

3 import Image from "next/image";

```

```

20 {data.user.image && (
21 <Image
22 src={data.user.image}
23 alt={data.user.fullname}
24 className={styles.navbar__user__image}
25 width={42}
26 height={42}
27 />
28)}

```

## KESIMPULAN

Pada praktikum ini mahasiswa telah:

- ✓ Mengimplementasikan Google OAuth
- ✓ Mengintegrasikan dengan NextAuth.js
- ✓ Menyimpan user ke Firestore

- ✓ Mengelola JWT dan Session
- ✓ Mengimplementasikan Multi-Role
- ✓ Menampilkan avatar profil